

Reviewer Pack — Minimal Rank of Geometric Invariants in Morse Theory vs Reso...

Entry 35bceb8c88ca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 12:05:41 UTC

Minimal Rank of Geometric Invariants in Morse Theory vs Resolution Proof Length for k-CNF

Entry ID: 35bceb8c88ca **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 12:05:41 UTC

1. Conjecture

Field A (mathematical branch): Morse Theory (Algebraic Topology) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for k-CNF

Statement:

[‘For every k-CNF formula F, the minimal rank of the Morse complex associated with its incidence graph is $\Omega(2^{n/3})$ where n is the number of variables in F.’, ‘Further, this lower bound holds only for expanders among all graphs corresponding to k-CNF formulas.’]

Rationale (proposer’s reasoning):

[‘Morse theory provides a way to study the topological structure of smooth manifolds, which can be applied to incidence graphs of k-CNF formulas. This geometric invariant captures information about the complexity of the resolution process for these formulas.’, ‘Previous studies have shown that expanders are particularly challenging for resolution algorithms, suggesting that Morse theory might expose structural properties relevant to resolution complexity.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 642df72d960bbf39

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average rank of Morse complexes from k-CNF formulas with up to 40 variables meets or exceeds a lower bound of $\Omega(2^{n/3})$, where n is the number of variables, and no single seed’s Morse complex rank exceeds 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Morse Theory AND complexity theory AND resolution proof length - geometric invariants Morse theory AND Resolution proof complexity k-CNF - minimal rank Morse complex $\Omega(2^{(n/3)})$ AND expanders k-CNF

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9201009v1] Large-Small Equivalence in String Theory - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2504.15729v2] Strong discrete Morse theory - [http://arxiv.org/abs/2401.17245v3] Geometric universal Jones invariant from configurations on ovals in the disc - [http://arxiv.org/abs/1803.04273v2] Morse index of multiplicity one min-max minimal hypersurfaces - [http://arxiv.org/abs/math/0406486v2] The Morse Complex for a Morse Function on a Manifold with Corners

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n):
        clauses = []
        for _ in range(n * 2):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            clauses.append(clause)
        return clauses

    def incidence_graph(clauses):
        graph = {i: set() for i in range(1, n + 1)}
        for clause in clauses:
            for literal in clause:
                if literal > 0:
                    graph[literal].add(-literal)
```

```

        else:
            graph[-literal].add(literal)
    return graph

def morse_complex(graph):
    rank = 0
    visited = set()
    stack = list(graph.keys())

    while stack:
        node = stack.pop()
        if node not in visited:
            rank += 1
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return rank

n = random.randint(5, 40)
clauses = generate_k_cnf(n)
graph = incidence_graph(clauses)
morse_rank = morse_complex(graph)

if morse_rank > 10:
    counterexample = f"Rank {morse_rank} exceeds 10 for n={n}"
    return {
        "metric_name": "Morse Rank",
        "metric_value": morse_rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": counterexample
    }

lower_bound = Fraction(2**(n/3)).limit_denominator()
if morse_rank < lower_bound:
    return {
        "metric_name": "Morse Rank",
        "metric_value": morse_rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Rank {morse_rank} below lower bound {lower_bound} for n={n}"
    }

return {
    "metric_name": "Morse Rank",
    "metric_value": morse_rank,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:

```

```

seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
total_rank = 0
count_conjecture_holds = 0

for seed in seeds:
    trial_result = run_trial(seed)
    results.append(trial_result)
    total_rank += trial_result["metric_value"]
    if trial_result["conjecture_holds"]:
        count_conjecture_holds += 1

mean_rank = Fraction(total_rank).limit_denominator() / len(results)
support_fraction = Fraction(count_conjecture_holds, len(results))

print("RESULT:", "SUPPORTED" if support_fraction >= Fraction(4, 5) else "FALSIFIED", f"mean={n

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dec656bf.py", line 101, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dec656bf.py", line 61, in run_trial
    morse_rank = morse_complex(graph)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dec656bf.py", line 52, in morse_complex
    for neighbor in graph[node]:
                   ~~~~~^~~~~~
KeyError: -33

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14089
2	preregistration	ollama_remote	glm4:latest	0	0	9427
3	novelty	ollama_remote	glm4:latest	0	0	8343
4	novelty	ollama_remote	glm4:latest	0	0	23873
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16357
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12428
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11352
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8890
9	judge	ollama_remote	glm4:latest	0	0	11545

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 116305 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/35bceb8c88ca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/35bceb8c88ca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/35bceb8c88ca.tar.gz` (if generated)